

Final Report

CU LibSpace

Digital seat reservation platform for library staff and students

Present to

Assoc. Prof. Chotirat Ratanamahatana

By Group No 7

6730289921	Prawanwit	Pimsawas
6730407621	Phattaraphum	Jamroensuk
6731315721	Natthakorn	Yiengyongphan
6731332321	Poonnawit	Supawasuwat
6731340321	Phurinut	Phanpaew
6731341021	Pusorn	Ngamsanlert
6731348421	Waramet	Limpanich
6731360921	Sirawit	Kaewpeng
6731365021	Aut	Suttitanamongkol

Term Project

2110322 Database Systems, Semester 2 of Academic year 2025

Department of Computer Engineering,
Faculty of Engineering, Chulalongkorn University

Table of Contents

Introduction_____	1
Organization background_____	1
Objective of the project_____	2
An overview of the system_____	2
System functionalities_____	3
ER Diagram (Chen's notation)_____	6
Schema Diagram with Referential Integrity_____	7
Normalization (3NF) of a Relation_____	8
Data Dictionary of Relations (Database Tables) and their Attributes_____	10
Database Implementation and SQL DML Operations with Referential Integrity Constraints_____	14
Identify the indexed relations and attributes_____	20
Stored Procedures_____	22
Stored Functions_____	27
Triggers_____	32
Execution Path_____	35
SQL Complex Query_____	39
Document-based Design Schema_____	44
Functions to demonstrate data manipulation operations for a NoSQL database using MongoDB Compass_____	46

Project name: CU LibSpace

Digital seat reservation platform for library staff and students

Introduction

Currently, the Office of Academic Resources at Chulalongkorn University (Central Library) serves as a primary facility for students preparing for examinations. However, the library frequently faces the issue of insufficient reading seats, particularly during Midterm and Final examination periods. Students often waste valuable time searching for available seats across different floors. Furthermore, the problem of "seat hogging"—where personal items are left to claim a seat without the user's actual presence—results in the inefficient utilization of library resources.

To address these challenges, the "CU LibSpace" project aims to transform the service model from a traditional walk-in basis to a digital reservation system. This platform allows users to book seats in advance, ensuring availability and reducing unfair reservation practices. Additionally, the system provides library administrators with the tools to manage space and resources more effectively.

Organization background

CU LibSpace operates as a centralized platform connecting Service Receivers (students, staff, and outsiders) with Service Providers (library resources). The system relies on two key Chulalongkorn University partners: the Office of Academic Resources (OAR) for facility management and policies, and the Office of the Registrar for CUNET user authentication.

To optimize resource allocation, CU LibSpace functions as a "Match Maker" system, directly pairing available seating with users in need. This digital transition aligns with the university's "Smart Campus" initiative, utilizing technology to efficiently manage high demand during peak periods.

Ultimately, the project benefits key stakeholders by empowering Library Staff to monitor usage and enforce regulations without physical confrontation. Furthermore, the University Administration gains data-driven insights to help plan future campus learning spaces.

Objective of the project

The primary objective of the CU LibSpace project is to resolve the chronic issue of insufficient reading seats and inefficient space utilization within the Office of Academic Resources, particularly during examination periods. By transitioning from a traditional walk-in service to a digital reservation system, the project aims to eliminate unfair practices such as "seat hogging" and guarantee seat availability for students. Furthermore, the system seeks to empower library administrators with real-time monitoring tools to manage zones and enforce usage discipline through an automated penalty system for "No-show" behaviors. Additionally, the project intends to streamline access for external users by implementing a ticketing system, allowing outside users to purchase tickets for library entry.

An overview of the system

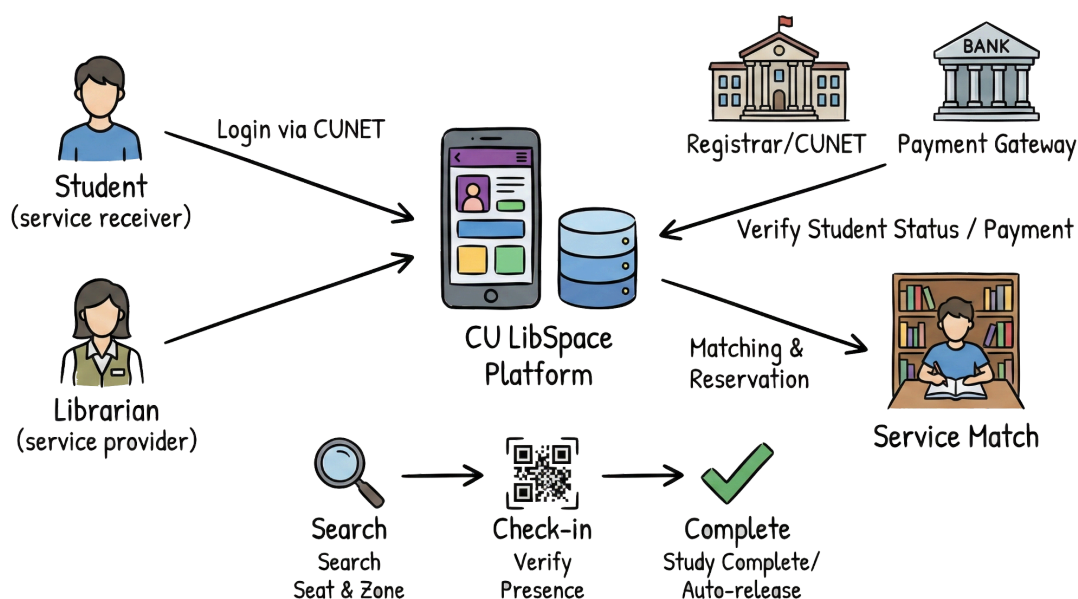
This application is designed to bring benefits to both Service Receivers (University Users/Outside Users) and Service Providers (Admins). Both parties will interact with the same platform, however, their interface and functionalities will differ based on their user type.

For Students to get started, they are required to log in via the CUNET authentication system. This process verifies their student status against the university registrar database, ensuring that only active students can access the student-exclusive reservation quotas without needing to manually upload documents. For Outsiders, they must register and purchase an access ticket to gain entry.

The reservation system (Match Making) starts when the user selects the desired Zone, Date, and Time slot. The system then displays available seats in real-time. Once a user confirms a booking, the system "holds" that seat, preventing other users from booking the same resource simultaneously.

Upon arrival (During the service), the user must perform a Check-in by scanning the QR code at their reserved desk. The application will then request location access to verify their physical presence within a specific timeframe. This step is crucial to prevent "seat hogging." If the user fails to check in, the system automatically cancels the booking and deducts Credit Scores as a penalty.

When the service is completed or the booking time expires, the user checks out, and the seat status automatically updates to "Available" for the next user. Users can also report issues regarding the facility directly through the app.



System functionalities

1. Register and Login System

- 1.1. The system shall allow students and staff to login via the CUNET authentication system to verify their status with the university registrar.
- 1.2. The system shall allow outsiders to create an account by providing their personal information, including full name, citizen ID (or passport number), phone number, and email address.
- 1.3. The system shall ensure that each User Account is uniquely identified by a User ID (UID).
- 1.4. The system shall allow librarians (admins) to login using their specific staff credentials.
- 1.5. The system shall allow users to update their profile information, such as name, phone number and email.

2. Search and Browse Seats

- 2.1. The system shall allow users to view the map or layout of the library zones.
- 2.2. The system shall allow users to see the status of each table in real-time (Available, Reserved, Occupied, or Closed).
- 2.3. The system shall allow users to filter tables based on criteria such as zone type (Silent, Group, Common), or facilities (plug).
- 2.4. The system shall allow users to check table availability for a specific date and time slot.

3. Book a Table (Match Making)

- 3.1. The system shall allow users to select a specific table (TableID) to make a reservation.
- 3.2. The system shall allow users to choose the StartDateTime and EndDateTime of the booking if the duration is under a time limit. The system shall prevent users from booking more than one table at the same time (1 table = 1 booking policy).
- 3.3. The system shall "hold" the selected table immediately upon confirmation to prevent double booking from other users.
- 3.4. The system shall record every reservation detail when a user makes a booking, including the exact Date, Time, and Timestamp of the transaction, as well as the BookingTIME.

4. Check-in and Verification

- 4.1. The system shall require users to perform a "Check-in" upon arrival to verify their physical presence.
- 4.2. The system shall record the ArriveTIME in the booking record when the user successfully checks in.
- 4.3. If the user fails to check in within the timeframe, the system shall automatically change the booking status to "No-show."
- 4.4. The system shall automatically release the "No-show" seat back to "Available" status for other users.

5. Cancellation and Penalty Management

- 5.1. The system shall allow users to cancel their reservation before the scheduled start time without any penalty.
- 5.2. In case of a "No-show" (failure to check in), the system shall automatically deduct the user's Credit Score as a penalty.
- 5.3. The system shall allow users to view their current BehaviourScore.
- 5.4. The system shall allow admins to manage and override user scores (MANAGE_SCORE) in case of system errors or appeals.

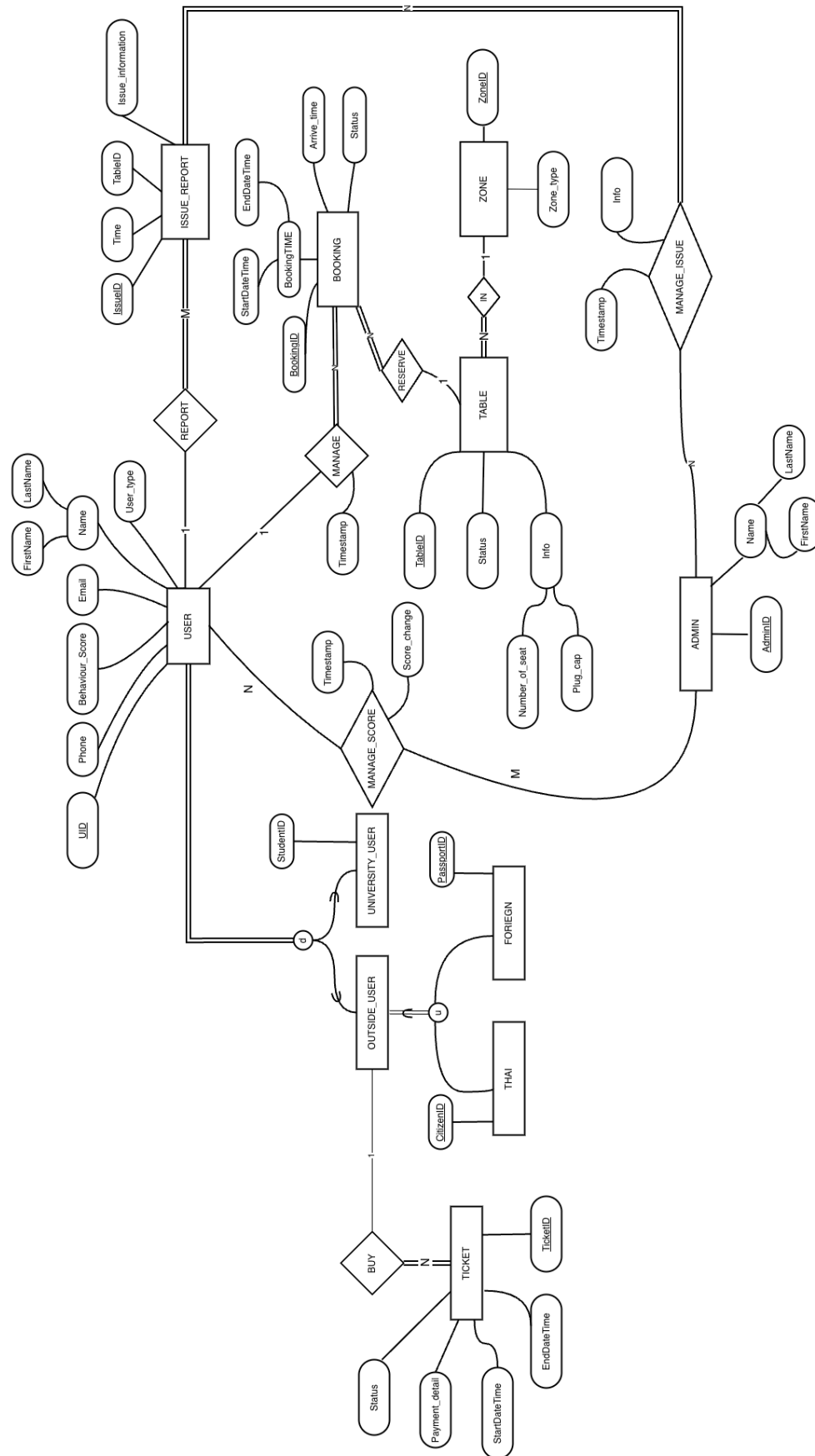
6. Ticket (For Outsiders)

- 6.1. The system shall allow outside users to pay for a ticket to access the service.
- 6.2. The system shall record the ticket details, including TicketID, Payment_detail, StartDateTime, and EndDateTime
- 6.3. The system shall update the Ticket's status to "Active" immediately after the ticket purchase is successful.

7. Admin and Facility Management (For Librarians)

- 7.1. The system shall allow users to report issues regarding the facility by submitting an `ISSUE_REPORT`.
- 7.2. The system shall require the user to provide specific details for the report, including `Issue_information`, `Time`, and optionally the related `TableID`.
- 7.3. The system shall allow admins to view all submitted issue reports.
- 7.4. The system shall allow admins to manage and resolve the reported issues (`MANAGE_ISSUE`).

ER Diagram (Chen's notation)



Schema Diagram with Referential Integrity

USER

<u>UID</u>	Phone	Behaviour_Score	Email	Firstname	Lastname	User_type
------------	-------	-----------------	-------	-----------	----------	-----------

functionOUTSIDE_USER

<u>UID</u>

UNIVERSITY_USER

<u>UID</u>	StudentID
------------	-----------

THAI

<u>UID</u>	CitizenID
------------	-----------

FOREIGN

<u>UID</u>	PassportID
------------	------------

ADMIN

<u>AdminID</u>	Firstname	Lastname
----------------	-----------	----------

ZONE

<u>ZoneID</u>	Zone_type
---------------	-----------

TABLE

<u>TableID</u>	Status	Number_of_seat	Plug_cap	ZoneID
----------------	--------	----------------	----------	--------

BOOKING

<u>BookingID</u>	StartDateTime	EndDateTime	Arrive_time	Status	UID	Timestamp	TableID
------------------	---------------	-------------	-------------	--------	-----	-----------	---------

TICKET

<u>TicketID</u>	Payment_detail	StartDateTime	Status	EndDateTime	UID
-----------------	----------------	---------------	--------	-------------	-----

ISSUE_REPORT

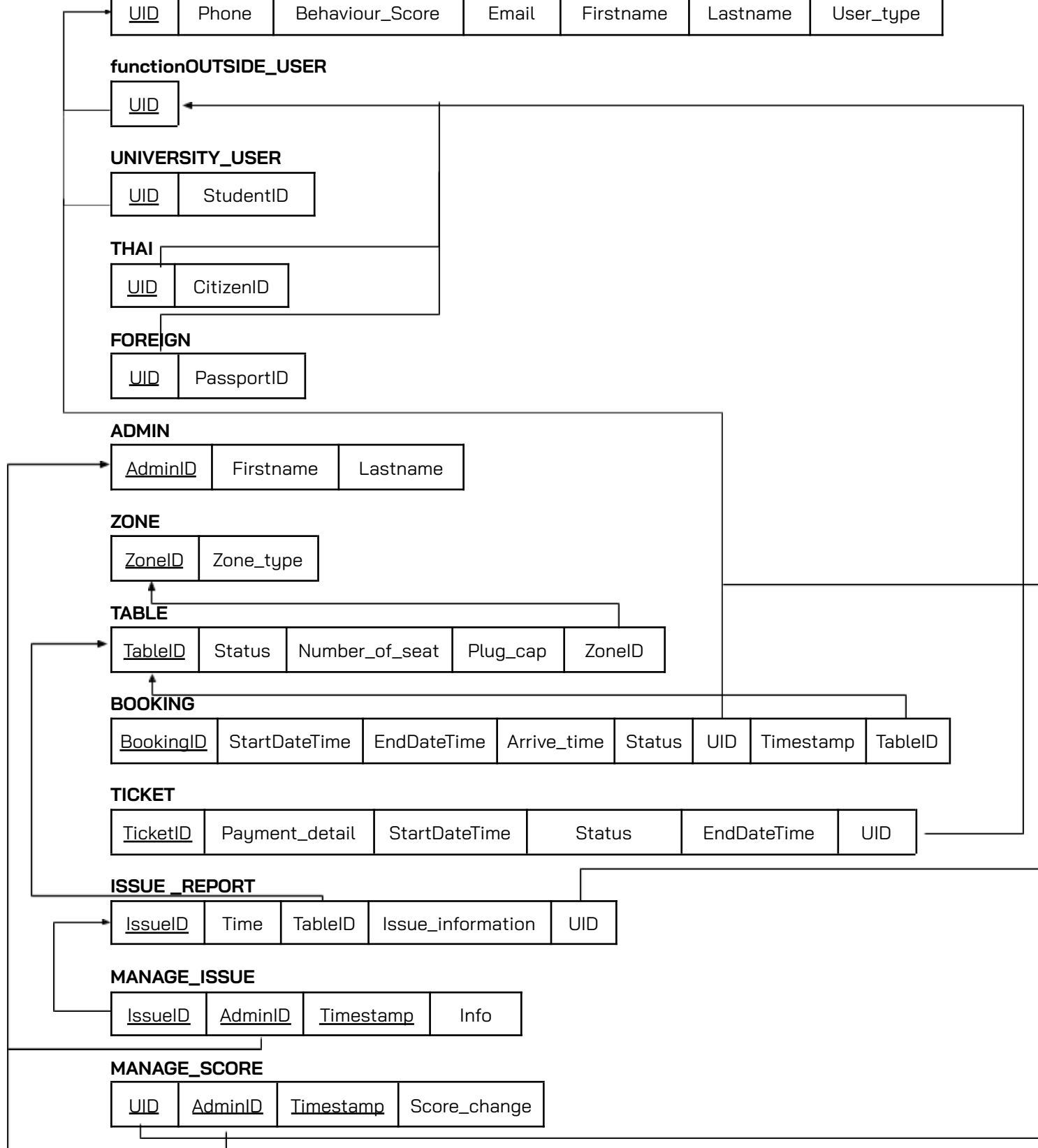
<u>IssueID</u>	Time	TableID	Issue_information	UID
----------------	------	---------	-------------------	-----

MANAGE_ISSUE

<u>IssueID</u>	<u>AdminID</u>	<u>Timestamp</u>	Info
----------------	----------------	------------------	------

MANAGE_SCORE

<u>UID</u>	<u>AdminID</u>	<u>Timestamp</u>	Score_change
------------	----------------	------------------	--------------



Normalization (3NF) of a Relation

USER

<u>UID</u>	Phone	Behaviour_Score	Email	Firstname	Lastname	User_type
------------	-------	-----------------	-------	-----------	----------	-----------

OUTSIDE_USER

<u>UID</u>

UNIVERSITY_USER

<u>UID</u>	StudentID
------------	-----------

THAI

<u>UID</u>	CitizenID
------------	-----------

FOREIGN

<u>UID</u>	PassportID
------------	------------

ADMIN

<u>AdminID</u>	Firstname	Lastname
----------------	-----------	----------

ZONE

<u>ZoneID</u>	Zone_type
---------------	-----------

TABLE

<u>TableID</u>	Status	Number_of_seat	Plug_cap	ZoneID
----------------	--------	----------------	----------	--------

BOOKING

<u>BookingID</u>	StartDateTime	EndDateTime	Arrive_time	Status	UID	Timestamp	TableID
------------------	---------------	-------------	-------------	--------	-----	-----------	---------

TICKET

<u>TicketID</u>	Payment_detail	Status	StartDateTime	EndDateTime	UID
-----------------	----------------	--------	---------------	-------------	-----

ISSUE_REPORT

<u>IssueID</u>	Time	TableID	Issue_information	UID
				

MANAGE_ISSUE

<u>IssueID</u>	<u>AdminID</u>	<u>Timestamp</u>	Info
			

MANAGE_SCORE

<u>UID</u>	<u>AdminID</u>	<u>Timestamp</u>	Score_change
			

Data Dictionary of Relations (Database Tables) and their Attributes

1. USER

Attribute Name	Type	Descriptive Name	Valid Values	Allow Nulls	Key
UID	INT	User Identifier	1, 2, 3, ...	No	PK
FirstName	VARCHAR(50)	User's first name	e.g. Somchai	No	-
LastName	VARCHAR(50)	User's last name	e.g. Yodrak	No	-
Phone	CHAR(10)	Phone number	e.g. 0812345678	No	-
Email	VARCHAR(100)	Email address	e.g. user@mail.com	Yes	-
Behaviour_Score	DECIMAL(4,1)	Behaviour score (0-100)	0.0 – 100.0	No	-
User_type	VARCHAR(20)	Type of user	THAI, FOREIGN, UNIVERSITY	No	-

2. ADMIN

Attribute Name	Type	Descriptive Name	Valid Values	Allow Nulls	Key
AdminID	INT	Admin Identifier	1, 2, 3, ...	No	PK
FirstName	VARCHAR(50)	Admin's first name	e.g. Malee	No	-
LastName	VARCHAR(50)	Admin's last name	e.g. Prasert	No	-

3. UNIVERSITY_USER

Attribute Name	Type	Descriptive Name	Valid Values	Allow Nulls	Key
UID	INT	User Identifier (FK to USER)	Existing UID	No	PK,FK
StudentID	CHAR(10)	University student/staff ID	e.g. 6601234567	No	-

4. OUTSIDE_USER

Attribute Name	Type	Descriptive Name	Valid Values	Allow Nulls	Key
UID	INT	User Identifier (FK to USER)	Existing UID	No	PK, FK

5. FOREIGN

Attribute Name	Type	Descriptive Name	Valid Values	Allow Nulls	Key
UID	INT	User Identifier (FK to OUTSIDE_USER)	Existing UID	No	PK,FK
PassportID	CHAR(9)	Passport number	e.g. AA1234567	No	-

6. THAI

Attribute Name	Type	Descriptive Name	Valid Values	Allow Nulls	Key
UID	INT	User Identifier (FK to OUTSIDE_USER)	Existing UID	No	PK,FK
CitizenID	CHAR(13)	Thai national ID number	e.g. 1100123456789	No	

7. ZONE

Attribute Name	Type	Descriptive Name	Valid Values	Allow Nulls	Key
ZoneID	INT	Zone Identifier	1, 2, 3, ...	No	PK
Zone_type	VARCHAR(30)	Type or name of zone	e.g. Quiet, Group	No	-

8. TABLE

Attribute Name	Type	Descriptive Name	Valid Values	Allow Nulls	Key
TableID	INT	Table Identifier	1, 2, 3, ...	No	PK
ZoneID	INT	Zone the table belongs to (FK to ZONE)	Existing ZoneID	No	FK
Status	VARCHAR(20)	availability status	Available, Reserved, Occupied, Closed	No	-
Number_of_seat	INT	Number of seats available	1 – 10	No	-
Plug_cap	INT	Number of plugcaps available	1, 2, 3, ...	Yes	-

9. BOOKING

Attribute Name	Type	Descriptive Name	Valid Values	Allow Nulls	Key
BookingID	INT	Booking Identifier	1, 2, 3, ...	No	PK
UID	INT	User who made the booking (FK to USER)	Existing UID	No	FK
TableID	INT	Table booked (FK to TABLE)	Existing TableID	No	FK
StartDateTime	DATETIME	Booking start date and time	e.g. 2025-04-12 09:00	No	-
EndDateTime	DATETIME	Booking end date and time	e.g. 2025-04-12 10:00	No	-
Arrive_Time	DATETIME	Actual arrival time of user	e.g. 2025-04-12 9.15	Yes	-
Status	VARCHAR(20)	Status of the booking	Pending, Active, Completed, Cancelled, No-show	No	-
Timestamp	TIMESTAMP	When booking was created	System timestamp	No	-

10. TICKET

Attribute Name	Type	Descriptive Name	Valid Values	Allow Nulls	Key
TicketID	INT	Ticket Identifier	1, 2, 3, ...	No	PK
UID	INT	User who owns the ticket (FK to OUTSIDE_USER)	Existing UID	No	FK
Payment_detail	VARCHAR(255)	Payment information	e.g. QR Code, Credit Card	Yes	-
Status	VARCHAR(20)	payment status	Paid, Unpaid	No	-
StartDate Time	DATETIME	Ticket validity start	e.g. 2025-04-01 00:00	No	-
EndDateTime	DATETIME	Ticket expiry date and time	e.g. 2025-04-30 23:59	No	-

11. ISSUE_REPORT

Attribute Name	Type	Descriptive Name	Valid Values	Allow Nulls	Key
IssueID	INT	Issue Identifier	1, 2, 3, ...	No	PK
UID	INT	User who reports the issue (FK to USER)	Existing UID	No	FK
Time	DATETIME	Time of issue report	System datetime	No	-
TableID	INT	Table with the issue (FK to TABLE)	Existing TableID	No	FK
Issue_information	TEXT	Description of the issue	Free text	No	-

12. MANAGE_ISSUE

Attribute Name	Type	Descriptive Name	Valid Values	Allow Nulls	Key
IssueID	INT	Issue being managed (FK to ISSUE_REPORT)	Existing IssueID	No	PK, FK
AdminID	INT	Admin managing the issue (FK to ADMIN)	Existing AdminID	No	PK, FK
Timestamp	TIMESTAMP	When management action occurred	System timestamp	No	PK
Info	TEXT	Description of the issue	Free text	No	-

13. MANAGE_SCORE

Attribute Name	Type	Descriptive Name	Valid Values	Allow Nulls	Key
UID	INT	User whose score is being managed (FK to USER)	Existing UID	No	PK,FK
AdminID	INT	Admin who managed the score (FK to ADMIN)	Existing AdminID	No	PK,FK
Timestamp	TIMESTAMP	When score was adjusted	System timestamp	No	PK
Score_change	INT	Amount of score adjusted	-100, -99, ..., +99, +100	No	-

Database Implementation and SQL DML Operations with Referential Integrity Constraints

- CREATE TABLES

```

CREATE TABLE "user" (
    UID          SERIAL          PRIMARY KEY,
    Phone        CHAR(10)        NOT NULL UNIQUE,
    Behaviour_Score DECIMAL(4,1) NOT NULL DEFAULT 100.0
                                CHECK (Behaviour_Score >= 0 AND
Behaviour_Score <= 100),
    Email        VARCHAR(100),
    Firstname    VARCHAR(50)     NOT NULL,
    Lastname     VARCHAR(50)     NOT NULL,
    User_type    VARCHAR(20)     NOT NULL
                                CHECK (User_type IN
('THAI', 'FOREIGN', 'UNIVERSITY'))
);

CREATE TABLE admin (
    AdminID      SERIAL          PRIMARY KEY,
    Firstname    VARCHAR(50)     NOT NULL,
    Lastname     VARCHAR(50)     NOT NULL
);

CREATE TABLE university_user (
    UID          INT             PRIMARY KEY
                                REFERENCES "user"(UID) ON DELETE
CASCADE,
    StudentID    CHAR(10)        NOT NULL UNIQUE
);

CREATE TABLE outside_user (
    UID          INT             PRIMARY KEY
                                REFERENCES "user"(UID) ON DELETE
CASCADE
);

CREATE TABLE thai (
    UID          INT             PRIMARY KEY
                                REFERENCES outside_user(UID) ON
DELETE CASCADE,
    CitizenID    CHAR(13)        NOT NULL UNIQUE
);

CREATE TABLE foreign_user (
    UID          INT             PRIMARY KEY
                                REFERENCES outside_user(UID) ON
DELETE CASCADE,
    PassportID   CHAR(9)         NOT NULL UNIQUE
);

```



```

CREATE TABLE zone (
    ZoneID        SERIAL          PRIMARY KEY,
    Zone_type     VARCHAR(30)     NOT NULL
);

CREATE TABLE "table" (
    TableID       SERIAL          PRIMARY KEY,
    ZoneID        INT            NOT NULL REFERENCES
zone(ZoneID),
    Status        VARCHAR(20)     NOT NULL DEFAULT 'Available'
CHECK (Status IN
('Available', 'Reserved', 'Occupied', 'Closed')),
    Number_of_seat INT          NOT NULL CHECK (Number_of_seat
BETWEEN 1 AND 10),
    Plug_cap      INT
);

CREATE TABLE booking (
    BookingID     SERIAL          PRIMARY KEY,
    UID           INT            NOT NULL REFERENCES
"user"(UID),
    TableID       INT            NOT NULL REFERENCES
"table"(TableID),
    StartDateTime TIMESTAMP      NOT NULL,
    EndDateTime   TIMESTAMP      NOT NULL,
    Arrive_time   TIMESTAMP,
    Status        VARCHAR(20)     NOT NULL DEFAULT 'Pending'
CHECK (Status IN
('Pending', 'Active', 'Completed', 'Cancelled', 'No-show')),
    Timestamp     TIMESTAMP      NOT NULL DEFAULT NOW()
);

CREATE TABLE ticket (
    TicketID      SERIAL          PRIMARY KEY,
    UID           INT            NOT NULL REFERENCES
outside_user(UID),
    Payment_detail VARCHAR(255),
    Status        VARCHAR(20)     NOT NULL CHECK (Status IN
('Paid', 'Unpaid')),
    StartDateTime TIMESTAMP      NOT NULL,
    EndDateTime   TIMESTAMP      NOT NULL
);

CREATE TABLE issue_report (
    IssueID       SERIAL          PRIMARY KEY,
    UID           INT            NOT NULL REFERENCES
"user"(UID),
    Time          TIMESTAMP      NOT NULL DEFAULT NOW(),
    TableID       INT            NOT NULL REFERENCES
"table"(TableID),
    Issue_information TEXT        NOT NULL
);

```

```

CREATE TABLE manage_issue (
    IssueID      INT              NOT NULL REFERENCES
issue_report(IssueID),
    AdminID      INT              NOT NULL REFERENCES admin(AdminID),
    Timestamp    TIMESTAMP        NOT NULL DEFAULT NOW(),
    Info         TEXT             NOT NULL,
    PRIMARY KEY (IssueID, AdminID, Timestamp)
);

CREATE TABLE manage_score (
    UID          INT              NOT NULL REFERENCES
"user"(UID),
    AdminID      INT              NOT NULL REFERENCES
admin(AdminID),
    Timestamp    TIMESTAMP        NOT NULL DEFAULT NOW(),
    Score_change INT              NOT NULL CHECK (Score_change
BETWEEN -100 AND 100),
    PRIMARY KEY (UID, AdminID, Timestamp)
);

```

a. INSERT data into the database

i. Valid Data Insertion

```

INSERT INTO admin (AdminID, Firstname, Lastname) VALUES
(1, 'Library', 'Master'), -- System Admin (Used by trigger
audit score)
(2, 'Somsak', 'Jaidee'); -- Second Admin for manage_issue

-- Zone
INSERT INTO zone (ZoneID, Zone_type) VALUES
(1, 'Silent'),
(2, 'Group'),
(3, 'Common');

-- Table
-- 101,102 → Silent (1 seat, w/ plugs)
-- 201,202 → Group (4 seats,w/ plugs)
-- 301 → Common (6 seats, w/ plugs)
-- 501 → Closed (For testing sp_ReserveTable if
table was closed)
INSERT INTO "table" (TableID, ZoneID, Status, Number_of_seat,
Plug_cap) VALUES
(101, 1, 'Available', 1, 2),
(102, 1, 'Available', 1, 2),
(201, 2, 'Available', 4, 4),
(202, 2, 'Available', 4, 4),
(301, 3, 'Available', 6, 6),
(501, 1, 'Closed', 1, 0); -- maintenance table

-- Users
-- UID 1 → UNIVERSITY, score 100 - Top User (Execution

```

```

Path + CQ Top Zone)
-- UID 2 → UNIVERSITY, score 100 - student (test
Execution Path: has reserved Silent before)
-- UID 3 → UNIVERSITY, score 40 - low point (test
sp_ReserveTable fail)
-- UID 4 → THAI (Outside), not have ticket - test trigger
block
-- UID 5 → FOREIGN (Outside), have ticket - test trigger
via (happy path)
-- UID 6 → UNIVERSITY, score 100 - Cancel King (test CQ
At-Risk)
INSERT INTO "user" (UID, Phone, Behaviour_Score, Email,
Firstname, Lastname, User_type) VALUES
(1, '0811111111', 100.0, 'alice@student.chula.ac.th',
'Alice', 'wonderland', 'UNIVERSITY'),
(2, '0822222222', 100.0, 'Bobby@student.chula.ac.th',
'Bobby', 'dekdee', 'UNIVERSITY'),
(3, '0833333333', 40.0, 'lowscore@student.chula.ac.th',
'Lowscore', 'Student', 'UNIVERSITY'),
(4, '0844444444', 100.0, 'thai.noticket@gmail.com',
'pomkonThai', 'NoTicket', 'THAI'),
(5, '0855555555', 100.0, 'foreign.hasticket@gmail.com',
'pomkonForeign', 'HasTicket', 'FOREIGN'),
(6, '0866666666', 100.0, 'cancelking@student.chula.ac.th',
'Cancel', 'King', 'UNIVERSITY');

-- University Users
INSERT INTO university_user (UID, StudentID) VALUES
(1, '6731315721'),
(2, '6731332321'),
(3, '6700000003'),
(6, '6700000006');

-- Outside Users
INSERT INTO outside_user (UID) VALUES (4), (5);

INSERT INTO thai (UID, CitizenID) VALUES
(4, '110000000004');

INSERT INTO foreign_user (UID, PassportID) VALUES
(5, 'P12345678');

-- Ticket
-- UID 5 has ticket period for 30 days (test trigger happy
path)
-- UID 4 no-ticket (test trigger block)
INSERT INTO ticket (TicketID, UID, Payment_detail, Status,
StartDateTime, EndDateTime) VALUES
(1, 5, 'Credit Card: **** 5678', 'Paid',
CURRENT_DATE,
CURRENT_DATE + INTERVAL '1 days');

```

ii. Invalid Data Insertion

```
Case 1: Referential Integrity Violation
-- Violate UNIQUE constraint (Duplicate Data Entry)
-- ERROR: duplicate key value violates unique constraint
"user_phone_key"

=====

INSERT INTO "user" (Phone, Behaviour_Score, Email, Firstname,
Lastname, User_type)
VALUES ('0811111111', 100.0, 'duplicate@mail.com', 'Dup',
'User', 'THAI');
-- ^ Phone '0811111111' repeat with Alice (UID=1) that
already has → ERROR
```

b. DELETE data from the database

i. Valid Data Deletion

```
-- Delete BOOKING that was cancelled
DELETE FROM booking
WHERE Status = 'Cancelled';

-- Delete TICKET that expired
DELETE FROM ticket
WHERE EndDateTime < NOW();
```

ii. Invalid Data Deletion

```
Case 2: Referential Integrity Violation
-- Delete USER that still has BOOKING referring
-- ERROR: update or delete on table "user" violates foreign
key constraint
-- on table "booking" – booking.UID where UID=2

=====

DELETE FROM "user" WHERE UID = 2;
-- ^ Bob (UID=2) still BookingID=2 (Status='Active') have
-- → ERROR because booking.UID which did not set ON DELETE
CASCADE
```

c. UPDATE data in the database

i. Valid Data Update

```
-- Save Check-in and change status from Booking to Active
UPDATE booking
SET Arrive_time = '2025-04-12 09:10',
    Status      = 'Active'
WHERE BookingID = 1;

-- Delete 5 scores if No-show
UPDATE "user"
SET Behaviour_Score = Behaviour_Score - 5
WHERE UID = 5;

-- Close table for maintenance
UPDATE "table"
SET Status = 'Closed'
WHERE TableID = 1;
```

ii. Invalid Data Update

```
-- c. UPDATE - Case 3: Referential Integrity Violation
-- UPDATE FK to TableID where is not in TABLE
-- ERROR: insert or update on table "booking" violates
foreign key constraint
--      "booking_tableid_fkey" - TableID 999 which is not
in TABLE
--
=====

UPDATE booking
SET TableID = 999
WHERE BookingID = 1;
-- ^ TableID 999 which is not in "table" → ERROR
```

Identify the indexed relations and attributes

BOOKING table

Attribute Name	Type	Descriptive Name	Valid Values	Allow Nulls	Key
BookingID	INT	Booking Identifier	1, 2, 3, ...	No	PK
UID	INT	User who made the booking (FK to USER)	Existing UID	No	FK
TableID	INT	Table booked (FK to TABLE)	Existing TableID	No	FK
StartDateTime	DATETIME	Booking start date and time	e.g. 2025-04-12 09:00	No	-
EndDateTime	DATETIME	Booking end date and time	e.g. 2025-04-12 10:00	No	-
ArriveTime	DATETIME	Actual arrival time of user	e.g. 2025-04-12 9.15	Yes	-
Status	VARCHAR(20)	Status of the booking	Pending, Active, Completed, Cancelled, No-show	No	-
Timestamp	TIMESTAMP	When booking was created	System timestamp	No	-

Indexing is implemented on the BOOKING table for the following:

1. UID: Implemented as a B-Tree index because it is used to query the booking history of a specific UID, which a B-Tree handles optimally. Furthermore, a B-Tree supports range queries if searching across specific time periods is required.
2. (TableID, Status): Implemented as a Composite B-Tree index because both columns are used to check table availability/status. Creating a composite index enables PostgreSQL to answer the query using a single index without having to scan multiple indexes.
3. (StartDateTime, EndDateTime): Implemented as a GiST index because it is used to verify booking time frames. GiST is particularly well-suited for range/interval data.

USER table

Attribute Name	Type	Descriptive Name	Valid Values	Allow Nulls	Key
UID	INT	User Identifier	1, 2, 3, ...	No	PK
FirstName	VARCHAR(50)	User's first name	e.g. Somchai	No	-
LastName	VARCHAR(50)	User's last name	e.g. Yodrak	No	-
Phone	CHAR(10)	Phone number	e.g. 0812345678	No	-
Email	VARCHAR(100)	Email address	e.g. user@mail.com	Yes	-
Behaviour_Score	DECIMAL(4,1)	Behaviour score (0-100)	0.0 – 100.0	No	-
User_type	VARCHAR(20)	Type of user	THAI, FOREIGN, UNIVERSITY	No	-

Indexing is implemented on the USER table for the following:

1. (FirstName, LastName): Implemented as a Composite B-Tree index because it is used to rapidly search for specific user data.

TABLE table

Attribute Name	Type	Descriptive Name	Valid Values	Allow Nulls	Key
TableID	INT	Table Identifier	1, 2, 3, ...	No	PK
ZoneID	INT	Zone the table belongs to (FK to ZONE)	Existing ZoneID	No	FK
Status	VARCHAR(20)	availability status	Available, Reserved, Occupied, Closed	No	-
Number_of_seat	INT	Number of seats available	1 – 10	No	-
Plug_cap	INT	Number of plugcaps available	1, 2, 3, ...	Yes	-

Indexing is implemented on the TABLE table for the following:

1. Status: Implemented as a Partial B-Tree index because users specifically need to search for tables where Status = 'Available'. Creating a partial index exclusively for rows that match this condition keeps the index smaller and makes queries faster.

Stored Procedures

1. Seat/Table Reservation -> sp_ReserveTable
 - a. The system first checks the requested start time (**p_start_time**) against the current system time (**CURRENT_TIMESTAMP**). If the user attempts to book a time slot in the past, the procedure immediately aborts and raises an exception.
 - b. The system retrieves the user's **Behaviour_Score** from the **USER** table. If the user's score has dropped below the minimum threshold of 50, they are restricted from making a reservation, and the system raises an exception detailing the failure.
 - c. Before checking the schedule, the system verifies the physical condition of the requested table in the **TABLE** table. If the table's status is currently marked as '**Closed**' (e.g., undergoing maintenance or broken), the booking is rejected.
 - d. To guarantee a seat is truly available, the procedure counts any existing reservations for that specific table that are either '**Pending**' or '**Active**'. It uses the **OVERLAPS** function to check if the requested time frame intersects with any established bookings. If an overlap is detected, the system denies the request to prevent double-booking.
 - e. Once the user and the time slot pass all validation checks, the system inserts a new reservation record into the **BOOKING** table. The booking status is explicitly set to '**Pending**' (meaning the user still needs to check in upon arrival), and the system logs the exact timestamp of the transaction. A success notice is then generated.
(Note: The table's physical status in the **TABLE** table remains unchanged during this step, as the user has not yet physically occupied the seat).

```
CREATE OR REPLACE PROCEDURE sp_ReserveTable(
  p_uid INT,
  p_table_id INT,
  p_start_time TIMESTAMP,
  p_end_time TIMESTAMP
)
LANGUAGE plpgsql
AS $$
DECLARE
  v_user_score DECIMAL(4,1);
  v_overlapping_count INT;
  v_table_status VARCHAR(20);
BEGIN
  -- 1. Validate the Booking Time (Time Logic)
  IF p_start_time < CURRENT_TIMESTAMP THEN
    RAISE EXCEPTION 'Booking failed: Cannot reserve a table in
```



```

the past';
    END IF;

    -- 2. Verify User Eligibility (Business Logic)
    SELECT Behaviour_Score INTO v_user_score FROM "user" WHERE UID
= p_uid;
    IF v_user_score < 50 THEN
        RAISE EXCEPTION 'Booking failed: Your behaviour score (%)
is below the required threshold', v_user_score;
    END IF;

    -- 3. Check the Physical Table Status (Physical Status Logic)
    SELECT Status INTO v_table_status FROM "table" WHERE TableID =
p_table_id;
    IF v_table_status = 'Closed' THEN
        RAISE EXCEPTION 'Booking failed: Table % is currently
closed for maintenance', p_table_id;
    END IF;

    -- 4. Prevent Double-Booking (Time-slot Logic)
    SELECT COUNT(*) INTO v_overlapping_count
    FROM BOOKING
    WHERE TableID = p_table_id
        AND Status IN ('Pending', 'Active')
        AND (p_start_time, p_end_time) OVERLAPS (StartDateTime,
EndDateTime);

    IF v_overlapping_count > 0 THEN
        RAISE EXCEPTION 'Booking failed: Table % is already booked
during the requested time', p_table_id;
    END IF;

    -- 5. Finalize and Record the Transaction
    INSERT INTO BOOKING (UID, TableID, StartDateime, EndDateime,
Status, Timestamp)
    VALUES (p_uid, p_table_id, p_start_time, p_end_time, 'Pending',
CURRENT_TIMESTAMP);
    RAISE NOTICE 'Booking successful: Table % from % to %',
p_table_id, p_start_time, p_end_time;
END;
$$;

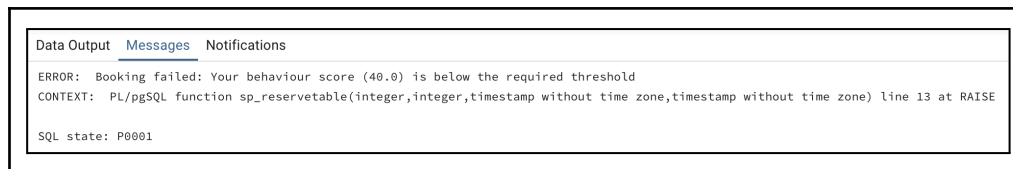
```

f. System Testing and Results

```

CALL sp_ReserveTable(
    3, 301,
    CURRENT_DATE + INTERVAL '2 days' + TIME '10:00',
    CURRENT_DATE + INTERVAL '2 days' + TIME '12:00'
); -- UID 3 (score 40) < 50
ERROR: Booking failed: Your behaviour score (40.0) is below
the required threshold

```



2. System for Check-in and Behaviour Score Penalty -> sp_CheckInUser

- a. The system first retrieves the reservation details from the **BOOKING** table using the provided booking ID. If the record does not exist, or if the booking's current status is anything other than '**Pending**' (such as already cancelled or completed), the procedure immediately aborts and raises an exception to prevent duplicate or invalid actions.
- b. The system then verifies if the user is attempting to check in too early. It compares the actual arrival time against the scheduled start time minus a 15-minute window. If the user scans the QR code before this allowed timeframe, the system denies the check-in and raises an exception indicating they must wait until their reservation approaches.
- c. Next, the system evaluates if the user has missed their reservation entirely. If the arrival time is more than 15 minutes past the scheduled start time, the system classifies the user as a no-show. It updates the booking status to '**No-show**', reverts the physical table's status back to '**Available**' so others can use it, logs a penalty in the **MANAGE_SCORE** table, and automatically deducts 10 points from the user's **Behaviour_Score**.
- d. Once the user successfully checks in within the acceptable 30-minute window (from 15 minutes before to 15 minutes after the start time), the system finalizes the arrival. The booking status is updated to 'Active' along with the exact arrival time, and the physical table's status in the **TABLE** table is changed to 'Occupied' to visually indicate to the system that the seat is now officially in use.

```
CREATE OR REPLACE PROCEDURE sp_CheckInUser(
    p_booking_id INT,
    p_arrive_time TIMESTAMP
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_start_time TIMESTAMP;
    v_uid INT;
    v_table_id INT;
    v_current_status VARCHAR(20);
BEGIN
    SELECT StartDateTime, UID, TableID, Status
```

```

    INTO v_start_time, v_uid, v_table_id, v_current_status
    FROM BOOKING
    WHERE BookingID = p_booking_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Booking record not found for ID %',
p_booking_id;
    END IF;

    -- Prevent checking in if the booking is not 'Pending'
    IF v_current_status != 'Pending' THEN
        RAISE EXCEPTION 'Check-in failed: Booking is currently %
and cannot be processed', v_current_status;
    END IF;

    -- [NEW FIX]: Prevent checking in TOO EARLY (e.g., more than 15
minutes before start time)
    IF p_arrive_time < (v_start_time - INTERVAL '15 minutes') THEN
        RAISE EXCEPTION 'Check-in failed: You are too early. You
can only check in up to 15 minutes before your reservation time.';
    END IF;

    -- Check if checking in TOO LATE (No-show logic)
    IF p_arrive_time > (v_start_time + INTERVAL '15 minutes') THEN

        UPDATE BOOKING
        SET Status = 'No-show', Arrive_time = p_arrive_time
        WHERE BookingID = p_booking_id;

        UPDATE "table" SET Status = 'Available' WHERE TableID =
v_table_id;

        UPDATE "user"
        SET Behaviour_Score = GREATEST(0, Behaviour_Score - 10)
        WHERE UID = v_uid;

        RAISE NOTICE 'Late check-in exceeding 15 minutes: Status
updated to No-show and behaviour score deducted successfully.';

    ELSE
        -- IF: Arrive in the correct time window (Success)
        UPDATE BOOKING
        SET Status = 'Active', Arrive_time = p_arrive_time
        WHERE BookingID = p_booking_id;

        UPDATE "table" SET Status = 'Occupied' WHERE TableID =
v_table_id;
        RAISE NOTICE 'Check-in successful: Welcome to the Central
Library space.';
    END IF;
END;
$$;

```

e. System Testing and Results

```
CALL sp_CheckInUser(555, NOW()::TIMESTAMP);  
-- BookingID 555 (UID 1, Table 101) → check-in ทันเวลา  
NOTICE: Check-in successful: Welcome to the Central Library  
space.
```

Data Output [Messages](#) Notifications

NOTICE: Check-in successful: Welcome to the Central Library space.
CALL

Query returned successfully in 35 msec.

```
CALL sp_CheckInUser(999, NOW()::TIMESTAMP);  
- No-show - BookingID 999 already pass by 30 minutes → more  
than 15 minutes  
NOTICE: Late check-in exceeding 15 minutes: Status updated  
to No-show and behaviour score deducted successfully.
```

Data Output [Messages](#) Notifications

NOTICE: Late check-in exceeding 15 minutes: Status updated to No-show and behaviour score deducted successfully.
CALL

Query returned successfully in 29 msec.

Stored Functions

1. Available Table Slots Retrieval -> `get_table_free_slots`
 - a. Define parameters: Accepts `library_open`, `library_close` for operating hours, `p_date` for the target date, and `p_now` as the reference timestamp (defaults to `NOW()`).
 - b. Filter occupied tables: The `all_tables` CTE excludes only `Closed` tables, allowing `Occupied` and `Reserved` tables to appear since they may still have free slots later in the day.
 - c. Generate boundary points: The `boundaries` CTE combines `v_slot_from`, booking `StartDateTime` and `EndDateTime`, and `v_day_close` into timestamp reference points per table. `UNION` removes duplicate timestamps caused by overlapping bookings.
 - d. Identify free slots: The `boundary_pairs` CTE uses `LEAD()` partitioned by `TableID` to pair adjacent boundary points into candidate slots. The `valid_slots` CTE keeps only slots of at least 30 minutes with no overlapping bookings via `NOT EXISTS`.
 - e. Validate start time: `v_slot_from` is set to `GREATEST(v_day_open, v_now)` to exclude past slots. The final `WHERE` clause additionally filters out slots ending before the next 30-minute boundary from `p_now`.
 - f. Return result set: Outputs `table_id` | `zone_type` | `number_of_seat` | `plug_cap` | `free_start` | `free_end` | `duration_min`, ordered by `zone_type`, `table_id`, and `free_start`.

```
CREATE OR REPLACE FUNCTION get_table_free_slots(
    library_open TIME DEFAULT '08:00',
    library_close TIME DEFAULT '20:00',
    p_date       DATE DEFAULT CURRENT_DATE,
    p_now        TIMESTAMP DEFAULT NOW()
)
RETURNS TABLE(
    table_id      INT,
    zone_type     VARCHAR,
    number_of_seat INT,
    plug_cap      INT,
    free_start    TIMESTAMP,
    free_end      TIMESTAMP,
    duration_min  INT
) AS $$
DECLARE
    v_day_open  TIMESTAMP;
    v_day_close TIMESTAMP;
```

```

v_now          TIMESTAMP := p_now;
v_slot_from    TIMESTAMP;
BEGIN
  v_day_open    := p_date + library_open;
  v_day_close   := p_date + library_close;

  IF v_day_close <= v_now AND p_date = CURRENT_DATE THEN
    RETURN;
  END IF;

  v_slot_from := CASE
    WHEN p_date = CURRENT_DATE THEN GREATEST(v_day_open, v_now)
    ELSE v_day_open
  END;

  RETURN QUERY
  WITH
    all_tables AS (
      SELECT t.TableID, z.Zone_type, t.Number_of_seat,
COALESCE(t.Plug_cap, 0) AS Plug_cap
      FROM "table" t JOIN zone z ON z.ZoneID = t.ZoneID
      WHERE t.Status != 'Closed'
    ),
    boundaries AS (
      SELECT at.TableID, v_slot_from AS ts FROM all_tables at
      UNION
      SELECT b.TableID, GREATEST(b.StartDateTime, v_slot_from)
      FROM booking b JOIN all_tables at ON at.TableID = b.TableID
      WHERE b.Status IN ('Pending', 'Active')
      AND b.StartDateTime < v_day_close AND b.EndDateTime >
v_slot_from
      UNION
      SELECT b.TableID, GREATEST(b.EndDateTime, v_slot_from)
      FROM booking b JOIN all_tables at ON at.TableID = b.TableID
      WHERE b.Status IN ('Pending', 'Active')
      AND b.StartDateTime < v_day_close AND b.EndDateTime >
v_slot_from
      UNION
      SELECT at.TableID, v_day_close FROM all_tables at
    ),
    boundary_pairs AS (
      SELECT TableID, ts AS slot_start,
      LEAD(ts) OVER (PARTITION BY TableID ORDER BY ts) AS
slot_end
      FROM boundaries
    ),
    valid_slots AS (
      SELECT bp.*
      FROM boundary_pairs bp
      WHERE bp.slot_end IS NOT NULL
      AND bp.slot_start < v_day_close
      AND bp.slot_end <= v_day_close

```

```

        AND (bp.slot_end - bp.slot_start) >= INTERVAL '30
minutes'
        AND NOT EXISTS (
            SELECT 1 FROM booking b
            WHERE b.TableID = bp.TableID
                AND b.Status IN ('Pending', 'Active')
                AND b.StartDateTime < bp.slot_end
                AND b.EndDateTime > bp.slot_start
        )
    )
    SELECT
        at.TableID, at.Zone_type, at.Number_of_seat, at.Plug_cap,
        vs.slot_start AS free_start,
        vs.slot_end AS free_end,
        EXTRACT(EPOCH FROM (vs.slot_end - vs.slot_start))::INT / 60
AS duration_min
    FROM valid_slots vs
    JOIN all_tables at ON at.TableID = vs.TableID
    WHERE (
        p_date + library_close <= p_now
        OR vs.slot_end > (
            DATE_TRUNC('hour', v_now) +
            INTERVAL '30 minutes' * CEIL(EXTRACT(MINUTE FROM v_now)
/ 30.0)
        )
    )
    ORDER BY at.Zone_type, at.TableID, vs.slot_start;
END;
$$ LANGUAGE plpgsql;

```

g. System Testing and Results

```

SELECT * FROM get_table_free_slots('08:00', '20:00',
CURRENT_DATE)
WHERE table_id = 202;
SELECT * FROM get_table_free_slots(
    '08:00',
    '20:00',
    CURRENT_DATE,
    (CURRENT_DATE + TIME '08:00:00')::TIMESTAMP
)
WHERE table_id = 202;
-- Table 202 today (BookingID 888 reserve at timeslot
13:00-15:00)

```

Data Output Messages Notifications								
	table_id	zone_type	number_of_seat	plug_cap	free_start	free_end	duration_min	
	integer	character varying	integer	integer	timestamp without time zone	timestamp without time zone	integer	
1	202	Group	4	4	2026-04-23 08:00:00	2026-04-23 13:00:00	300	
2	202	Group	4	4	2026-04-23 15:00:00	2026-04-23 20:00:00	300	

2. Zone Availability Summary -> get_zone_availability
 - a. Retrieve availability summary: A TABLE-returning function designed to summarize the number of available tables in each zone during a specified time period.
 - b. Return result set: Outputs a table displaying **ZoneID, Zone_type, the total number of tables, the number of booked tables, and the number of available tables.**
 - c. System application: Highly suitable for integration into the Search & Browse page of the system.

```

CREATE OR REPLACE FUNCTION get_zone_availability(
  p_start TIMESTAMP,
  p_end TIMESTAMP
)
RETURNS TABLE (
  zone_id          INT,
  zone_type        VARCHAR(30),
  total_tables     BIGINT,
  reserved_tables  BIGINT,
  available_tables BIGINT
) AS $$
BEGIN
  RETURN QUERY
  SELECT
    Z.ZoneID,
    Z.Zone_type,
    COUNT(DISTINCT T.TableID) AS total_tables,
    COUNT(DISTINCT B.TableID) AS reserved_tables,
    COUNT(DISTINCT T.TableID) - COUNT(DISTINCT B.TableID) AS
available_tables
  FROM ZONE Z
  JOIN "table" T ON T.ZoneID = Z.ZoneID AND T.Status != 'Closed'
  LEFT JOIN BOOKING B
    ON B.TableID = T.TableID
    AND B.Status IN ('Pending', 'Active')
    AND B.StartDateTime < p_end
    AND B.EndDateTime > p_start
  GROUP BY Z.ZoneID, Z.Zone_type
  ORDER BY Z.ZoneID;
END;
$$ LANGUAGE plpgsql;

```


d. System Testing and Results

```
SELECT * FROM get_zone_availability(  
    (CURRENT_DATE + TIME '13:00:00')::TIMESTAMP,  
    (CURRENT_DATE + TIME '15:00:00')::TIMESTAMP  
);
```

Data Output

Messages

Notifications

<

Triggers

1. Reservation Restriction for Outside Users (trg_check_ticket_for_outsider)
 - a. Description: This trigger executes BEFORE INSERT on the BOOKING table to verify if the requester is an Outside User. If so, they must possess a ticket with a 'Paid' status that fully covers the requested reservation period. If no active ticket is found, the system will block the reservation.

```
-- 1. Trigger Function: The logic to validate tickets
CREATE OR REPLACE FUNCTION fn_check_ticket_for_outsider()
RETURNS TRIGGER AS $$
DECLARE
    v_user_type VARCHAR(20);
BEGIN
    -- Retrieve the user type based on the UID of the new booking
    record
    SELECT User_type INTO v_user_type FROM "user" WHERE UID =
    NEW.UID;

    -- Check if the user is an Outside User (THAI or FOREIGN)
    IF v_user_type IN ('THAI', 'FOREIGN') THEN
        -- Verify if a 'Paid' ticket exists that covers the entire
        booking duration
        IF NOT EXISTS (
            SELECT 1 FROM ticket
            WHERE UID = NEW.UID
            AND Status = 'Paid'
            AND NEW.StartDateTime BETWEEN StartDateTime AND
            EndDateTime
            AND NEW.EndDateTime BETWEEN StartDateTime AND
            EndDateTime
        ) THEN
            -- Block the transaction if no valid ticket is found
            RAISE EXCEPTION 'Outside user (UID=%) must have an
            active ticket covering the booking period', NEW.UID;
        END IF;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- 2. Trigger: The event listener bound to the booking table
CREATE TRIGGER trg_check_ticket_for_outsider
BEFORE INSERT ON booking
FOR EACH ROW EXECUTE FUNCTION fn_check_ticket_for_outsider();
```

b. System Testing and Results

```
INSERT INTO booking (UID, TableID, StartDateTime,
EndDateTime)
VALUES (4, 202,
        CURRENT_DATE + INTERVAL '2 days' + TIME '10:00',
        CURRENT_DATE + INTERVAL '2 days' + TIME '12:00');
-- UID 4 (THAI, No ticket) Try to reserve
ERROR: Outside user (UID=4) must have an active ticket
covering the booking period
```

Data Output [Messages](#) Notifications

ERROR: Outside user (UID=4) must have an active ticket covering the booking period
CONTEXT: PL/pgSQL function fn_check_ticket_for_outsider() line 15 at RAISE

SQL state: P0001

2. Score Adjustment Audit Log -> trg_audit_score_change

- a. Description: This trigger is designed as an Audit Log for score adjustments. Whenever an Administrator (Admin) manually modifies a behavior score in the **USER** table, or when an automated penalty occurs, this trigger executes AFTER UPDATE. It automatically records the transaction history in the **MANAGE_SCORE** table, documenting the previous score, the new score, the responsible party, and the precise timestamp of the change.

```
CREATE OR REPLACE FUNCTION fn_audit_score_change()
RETURNS TRIGGER AS $$
BEGIN
    IF OLD.Behaviour_Score <> NEW.Behaviour_Score THEN
        INSERT INTO manage_score (UID, AdminID, Timestamp,
Score_change)
        VALUES (
            NEW.UID,
            1,
            -- System Admin
            NOW(),
            ROUND(NEW.Behaviour_Score -
OLD.Behaviour_Score)::INT
        );
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_audit_score_change
AFTER UPDATE ON "user"
FOR EACH ROW EXECUTE FUNCTION fn_audit_score_change();
```

b. System Testing and Results

```
SELECT * FROM manage_score WHERE UID = 1 ORDER BY Timestamp;
-- Check if after Bookingid 999 come late (No-show → lower
point UID 1) manage_score is logged automatically
```

Data Output Messages Notifications				
Showing rows: 1 to 1 Page No: 1				
uid [PK] integer	adminid [PK] integer	timestamp [PK] timestamp without time zone	score_change integer	
1	1	2026-04-23 01:45:48.237225	-10	


```

      Index Cond: (uid = b.uid)
-> Index Scan using table_pkey on "table" t (cost=0.15..4.83 rows=1 width=8)
      Index Cond: (tableid = b.tableid)
-> Index Scan using zone_pkey on zone z (cost=0.15..0.23 rows=1 width=4)
      Index Cond: (zoneid = t.zoneid)
      Filter: ((zone_type)::text = 'Silent'::text)
(20 rows)

```

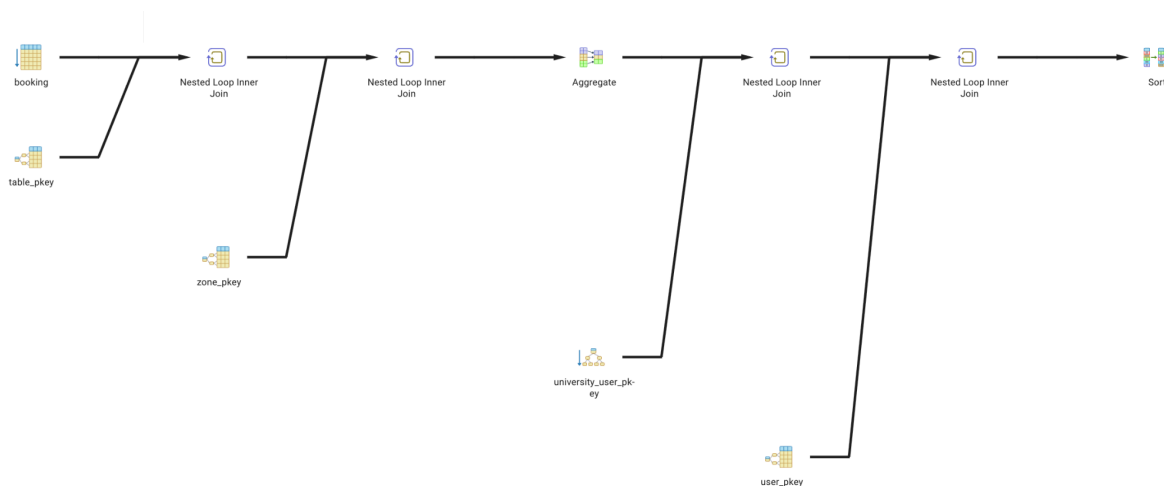
- Query Structure and Performance Analysis:
 - Execution Strategy: This structure utilizes Nested Loop and Hash Join operations to join all involved tables initially.
 - The Redundancy Issue: The primary drawback occurs if a single student has reserved a table in this zone multiple times (e.g., 10 times); the Join operation will generate 10 duplicate rows for that individual.
 - Resource Consumption: To resolve these duplicates, the system must pass the entire result set through a Sort Node followed by a Unique (or HashAggregate) operation in the final stage.
 - Overhead: Performing a Sort on a large volume of data requires high memory (RAM) resources and increases the overall Cost of the Execution Path.

Variation 2: Utilizing EXISTS for Semi-Join Logic

```

SELECT u.Firstname, u.Lastname
FROM "user" u
JOIN university_user uu ON u.UID = uu.UID
WHERE u.Behaviour_Score = 100.0
  AND EXISTS (
    SELECT 1
    FROM booking b
    JOIN "table" t ON b.TableID = t.TableID
    JOIN zone z ON t.ZoneID = z.ZoneID
    WHERE b.UID = u.UID
      AND z.Zone_type = 'Silent'
      AND (b.status = 'Completed' OR b.status = 'Active')
  )
ORDER BY u.Firstname ASC;

```



QUERY PLAN

```

Sort (cost=47.10..47.12 rows=8 width=236)
  Sort Key: u.firstname
  -> Nested Loop (cost=42.15..46.98 rows=8 width=236)
    -> Nested Loop (cost=42.00..46.70 rows=1 width=8)
      -> HashAggregate (cost=41.85..41.86 rows=1 width=4)
        Group Key: b.uid
        -> Nested Loop (cost=19.52..41.85 rows=1 width=4)
          -> Hash Join (cost=19.37..40.43 rows=6 width=8)
            Hash Cond: (t.tableid = b.tableid)
            -> Seq Scan on "table" t (cost=0.00..18.00 rows=800 width=8)
            -> Hash (cost=19.30..19.30 rows=6 width=8)
              -> Seq Scan on booking b (cost=0.00..19.30 rows=6 width=8)
                Filter: (((status)::text = 'Completed'::text) OR ((status)::text =
'Active'::text))
          -> Index Scan using zone_pkey on zone z (cost=0.15..0.23 rows=1 width=4)
            Index Cond: (zoneid = t.zoneid)
            Filter: ((zone_type)::text = 'Silent'::text)
        -> Index Only Scan using university_user_pkey on university_user uu (cost=0.15..4.84
rows=1 width=4)

```

Index Cond: (uid = b.uid)
 -> Index Scan using user_pkey on "user" u (cost=0.14..0.21 rows=1 width=240)
 Index Cond: (uid = uu.uid)
 Filter: (behaviour_score = 100.0)
 (21 rows)

- Query Structure and Performance Analysis:
 - Execution Strategy: This structure shifts from a direct JOIN to an EXISTS condition to verify data existence.
 - Optimized Execution Plan: According to the Execution Plan, the PostgreSQL Query Optimizer selects a HashAggregate node to group user IDs (**b.uid**) within the Booking table at the Subquery level.
 - No Row Multiplication: This approach prevents the multiplication of rows before the final retrieval.
 - Efficiency Gains: Consequently, the system avoids the need for Sort and Unique operations in the final stage, significantly reducing the computational overhead and memory usage.

System Testing and Results

Data Output			Messages	Notifications
≡ 📄 ▼ 📋 ▼ 🗑️ 📦 ⬇️ 📈 SQL			Showing rows: 1 to 1 ✎ 🖨️ Page No: 1	
1	firstname character varying (50)	lastname char	Graph Visualiser	
1	Bobby	dekdee		

SQL Complex Query

1. System for Identifying "At-Risk" Users for Potential Banning
 - a. Description: This function retrieves and summarizes the history of all users to identify "at-risk" behavior that may warrant a system ban.
 - b. Logic & Criteria: The system aggregates data to determine:
 - i. Reservation Default Count: The total number of times a user has failed to check in (**No-show**) or has **Cancelled** a booking.
 - ii. Total Penalty Points: The cumulative behavior points deducted by an Admin (recorded in the score management history).
 - c. Filtering Conditions: The report specifically filters for users who meet at least one of the following criteria:
 - i. Current **Behaviour_Score** is less than 100.
 - ii. Total "**No-show**" or "**Cancel**" count is 1 or more.

```

SELECT
  u.UID,
  u.Firstname || ' ' || u.Lastname AS Fullname,
  u.User_type,
  u.Behaviour_Score,
  COALESCE(bad_bookings.fail_count, 0) AS
total_failed_bookings,
  COALESCE(score_deductions.total_deducted, 0) AS
total_score_deducted
FROM "user" u
LEFT JOIN (
  SELECT UID, COUNT(BookingID) AS fail_count
  FROM booking
  WHERE Status IN ('No-show', 'Cancelled')
  GROUP BY UID
) bad_bookings ON u.UID = bad_bookings.UID
LEFT JOIN (
  SELECT UID, SUM(Score_change) AS total_deducted
  FROM manage_score
  WHERE Score_change < 0
  GROUP BY UID
) score_deductions ON u.UID = score_deductions.UID
WHERE u.Behaviour_Score < 100.0
  OR COALESCE(bad_bookings.fail_count, 0) >= 1
ORDER BY u.Behaviour_Score ASC, total_failed_bookings DESC;

```

d. System Testing and Results

```
-- UID 3 (score 40, deducted -60)
-- UID 1 (score 90 หลัง No-show, failed 1)
-- UID 6 (score 100, Appears in the list due to having 3
Failed Bookings (2 No-shows + 1 Cancellation). Even though
the Behaviour_Score remains at 100.0-as these are legacy
records that did not process through the automated deduction
procedure-our.)
```

Data Output Messages Notifications							
Showing rows: 1 to 3 Page No: 1							
	uid [PK] integer	fullname text	user_type character varying (20)	behaviour_score numeric (4,1)	total_failed_bookings bigint	total_score_deducted bigint	
1	3	Lowscore Stud...	UNIVERSITY	40.0	0	-60	
2	1	Alice wonderla...	UNIVERSITY	90.0	1	-10	
3	6	Cancel King	UNIVERSITY	100.0	3	0	

2. Top University User per Zone (Monthly Summary)

- a. Description: This report identifies the **UNIVERSITY_USER** with the highest number of bookings in each specific zone for every month of a given year (e.g., 2025).
- b. Ranking Logic: The system calculates the total booking count per user, per zone, per month.
 - i. It filters the results to display only the top-ranked user (Rank 1) for each zone.
- c. Data Constraints:
 - i. User Filtering: Only users belonging to the **UNIVERSITY_USER** subtype are considered.
 - ii. Handling Null Data: If a zone has no bookings during a particular month, that zone is excluded from the report for that month.
- d. Technical Implementation: * Window Function: Utilizes **RANK()** or **DENSE_RANK()** partitioned by **ZoneID** and the month of **StartDateTime**, ordered by the booking count in descending order.
 - i. Date Extraction: Uses **EXTRACT(MONTH FROM StartDateTime)** and **EXTRACT(YEAR FROM StartDateTime)** to group data chronologically.
 - ii. Joins: Performs joins between booking, user, and zone tables to map the user details and zone types.

```
WITH MonthlyZoneBooking AS (
  SELECT
```

```

        z.Zone_type,
        u.UID,
        u.Firstname,
        u.Lastname,
        COUNT(b.BookingID) AS TotalBookings
    FROM booking b
    JOIN university_user uu ON b.UID = uu.UID
    JOIN "user" u ON uu.UID = u.UID
    JOIN "table" t ON b.TableID = t.TableID
    JOIN zone z ON t.ZoneID = z.ZoneID
    WHERE EXTRACT(MONTH FROM b.StartDateTime) = 4
        AND EXTRACT(YEAR FROM b.StartDateTime) = 2025
    GROUP BY z.Zone_type, u.UID, u.Firstname, u.Lastname
),
RankedBookings AS (
    SELECT
        Zone_type,
        Firstname,
        Lastname,
        TotalBookings,
        RANK() OVER (PARTITION BY Zone_type ORDER BY
TotalBookings DESC) as Ranking
    FROM MonthlyZoneBooking
)

SELECT
    Zone_type,
    Firstname,
    Lastname,
    TotalBookings
FROM RankedBookings
WHERE Ranking = 1
ORDER BY Zone_type ASC;

```

e. System Testing and Results

```

-- Silent: UID 1 (2 Booking)
-- Group:  UID 1 (1 Booking), UID 2 (1 Booking), UID 6 (1
Booking)
-- Common: UID 1 (1 Booking)

```

Data Output Messages Notifications				
	zone_type character varying (30)	firstname character varying (50)	lastname character varying (50)	totalbookings bigint
1	Common	Alice	wonderland	2
2	Group	Alice	wonderland	1
3	Group	Bobby	dekdee	1
4	Group	Cancel	King	1
5	Silent	Alice	wonderland	3

3. Zone Usage and Booking Failure Analysis Summary

- a. Description: This report provides a comprehensive summary of zone popularity and operational issues. It evaluates the performance of each area (**Silent**, **Group**, **Common**) by comparing total reservation volume against "problematic" bookings to identify areas requiring spatial or policy improvements.
- b. Key Metrics:
 - i. Total Bookings: The cumulative number of reservations made in each zone to measure overall popularity and demand.
 - ii. Failed Bookings (Operational Issues): The combined total of bookings with a status of '**Cancelled**' or '**No-show**'.
 - iii. Failure Rate: A calculated percentage representing the ratio of failed bookings to total bookings, used to pinpoint which zone experiences the most frequent disruptions.

```

SELECT
    zone_summary.Zone_type,
    SUM(zone_summary.total_bookings) AS total_all_bookings,
    SUM(zone_summary.failed_bookings) AS
total_failed_bookings,
    (SUM(zone_summary.failed_bookings) * 100.0 /
NULLIF(SUM(zone_summary.total_bookings), 0)) AS
failed_percentage
FROM (
    SELECT
        z.Zone_type,
        t.TableID,
        COUNT(b.BookingID) AS total_bookings,
        SUM(CASE WHEN b.Status IN ('Cancelled', 'No-show')
THEN 1 ELSE 0 END) AS failed_bookings
    FROM zone z
    JOIN "table" t ON z.ZoneID = t.ZoneID
    LEFT JOIN booking b ON t.TableID = b.TableID
    GROUP BY z.Zone_type, t.TableID
) AS zone_summary
GROUP BY zone_summary.Zone_type
ORDER BY total_all_bookings DESC;

```

c. System Testing and Results

```

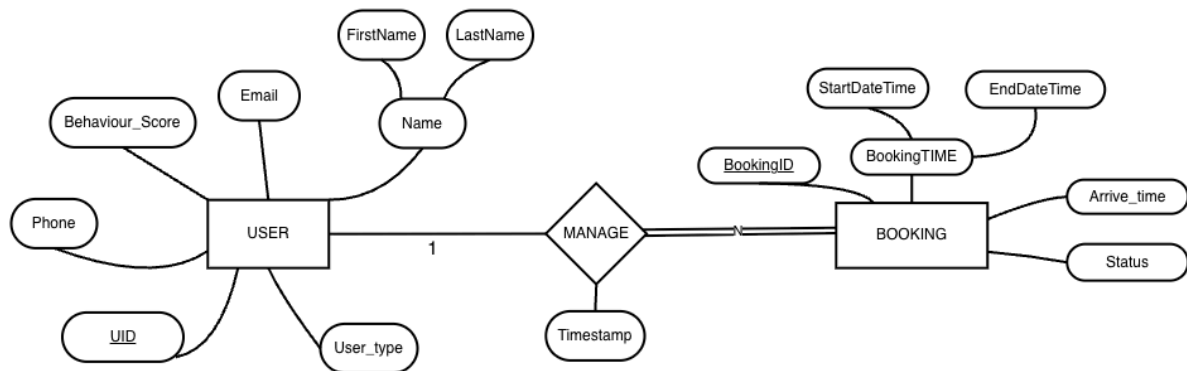
--      Silent: bookings from UID 1 (3 Completed) + UID 2 (1
Completed) + UID 6 (2 No-show)
--          = 6 total, 2 failed → ~33%
--      Group:  UID 1 (No-show 999), UID 2 (Pending 888), UID 6
(Cancelled 63)

```

```
--          = 4 total, 2 failed → 50%
--      Common: UID 1 (Pending 777 + 1A), UID 2 (Double-booking
1D = error -> didn't insert)
--          = 2 total, 0 failed
```

Data Output Messages Notifications				
Showing rows: 1 to 3 Page No: 1				
	zone_type character varying (30)	total_all_bookings numeric	total_failed_bookings numeric	failed_percentage numeric
1	Silent	6	2	33.3
2	Group	4	2	50.0
3	Common	2	0	0.0

Document-based Design Schema



The following section illustrates the transformation of the Relational ER Diagram into a Document-based Design Schema. The schema is represented in JSON format to showcase the denormalized structure and hierarchical data relationships inherent in the document model:

```

{
  "title": "user",
  "required": [
    "_id",
    "UID",
    "Firstname",
    "Lastname",
    "Phone",
    "User_type"
  ],
  "properties": {
    "_id": { "bsonType": "objectId" },
    "UID": { "bsonType": "int" },
    "Firstname": { "bsonType": "string" },
    "Lastname": { "bsonType": "string" },
    "Phone": {
      "bsonType": "string",
      "pattern": "^[0-9]{10}$"
    },
    "Email": { "bsonType": "string" },
    "Behaviour_Score": { "bsonType": "double" },
    "User_type": { "bsonType": "string" },
    "bookings": {
      "bsonType": "array",
      "items": {
        "bsonType": "object",
        "properties": {
          "BookingID": { "bsonType": "int" },
          "TableID": { "bsonType": "int" },
          "StartDateTime": { "bsonType": "date" },
          "EndDateTime": { "bsonType": "date" },
        }
      }
    }
  }
}
  
```

```
    "Arrive_time": { "bsonType": "date" },  
    "Status": { "bsonType": "string" }  
  }  
}  
}
```

Functions to demonstrate data manipulation operations for a NoSQL database using MongoDB Compass

- Function: getBadUsersReport()
 - Description: This function generates an audit report to identify users with "at-risk" behavior. It analyzes the reservation history within the bookings collection to pinpoint users with the highest frequency of No-shows or Cancellations.
 - NoSQL Implementation Strategy: * Aggregation Framework: Utilizes the **\$lookup** stage to join the users and bookings collections (if normalized) or performs an **\$aggregate** directly on embedded booking arrays.
 - Filtering: Uses the **\$match** stage to filter for statuses specifically marked as 'Cancelled' or 'No-show'.
 - Grouping: Employs the **\$group** stage by **UID** to calculate the total count of failed reservations per user.
 - Sorting: Applies the **\$sort** stage in descending order to highlight the most frequent offenders at the top of the report.
 - Operational Goal: To provide administrators with a clear list of users violating usage policies, enabling data-driven decisions for potential account restrictions or behavior score adjustments.

```
function getBadUsersReport() {
  return db.user.aggregate([
    {
      $addFields: {
        total_failed_bookings: {
          $size: {
            $filter: {
              input: "$bookings",
              as: "b",
              cond: { $in: ["$$b.Status", ["No-show",
"Cancelled"]] }
            }
          }
        },
        total_success_bookings: {
          $size: {
            $filter: {
              input: "$bookings",
              as: "b",
              cond: { $eq: ["$$b.Status", "Completed"] }
            }
          }
        }
      }
    }
  ])
}
```



```

        }
    },
    {
        $match: {
            $or: [
                { Behaviour_Score: { $lt: 100 } },
                { total_failed_bookings: { $gte: 1 } }
            ]
        },
    },
    {
        $project: {
            _id: 0,
            UID: 1,
            Fullname: { $concat: ["$Firstname", " ", "$Lastname"] },
            User_type: 1,
            Behaviour_Score: 1,
            total_failed_bookings: 1,
            total_success_bookings: 1
        }
    },
    {
        $sort: {
            total_failed_bookings: -1,
            Behaviour_Score: 1
        }
    }
    );
}

```

MongoDB Compass Result

Compass

My Queries

Data Modeling

CONNECTIONS (2)

Search connections

cluster0.oe4kfy.mongodb.net

localhost:27017

- admin
 - temproles
 - tempusers
- config
 - settings
- culibspace
 - user**
- local
 - replsetelection
 - replsetminvalid
 - startup_log
- test
 - post
 - postl
 - post_w_options

localhost:27017 > culibspace > user

Documents 4 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

[EXPLAIN](#) [RESET](#) [FIND](#) [OPTIONS](#)

[ADD DATA](#) [UPDATE](#) [DELETE](#) [EXPORT DATA](#) [EXPORT CODE](#)

25 1-4 of 4

```
{ "_id": ObjectId('69e5e739bcd483369fd339dd'),
  "UID": 1,
  "Firstname": "Alice",
  "Lastname": "Smith",
  "Phone": "0811111111",
  "Email": "alice@student.chula.ac.th",
  "Behaviour_Score": 100,
  "User_type": "UNIVERSITY",
  "bookings": Array (1) }
```

```
{ "_id": ObjectId('69e5e739bcd483369fd339de'),
  "UID": 2,
  "Firstname": "Bob",
  "Lastname": "Jones",
  "Phone": "0822222222",
  "Email": "bob@student.chula.ac.th",
  "Behaviour_Score": 100,
  "User_type": "UNIVERSITY",
  "bookings": Array (1) }
```

```
{ "_id": ObjectId('69e5e739bcd483369fd339df'),
  "UID": 3,
  "Firstname": "Carol",
  "Lastname": "Lee",
  "Phone": "0833333333",
  "Email": "carol@mail.com",
  "Behaviour_Score": 100,
  "User_type": "TRAI",
  "bookings": Array (1) }
```

```
{ "_id": ObjectId('69e5e739bcd483369fd339e0'),
  "UID": 5,
  "Firstname": "Eve",
  "Lastname": "Wilson",
  "Phone": "0855555555",
  "Email": "eve@student.chula.ac.th",
  "Behaviour_Score": 100,
  "User_type": "TRAI",
  "bookings": Array (1) }
```

Compass

My Queries

Data Modeling

CONNECTIONS (2)

Search connections

cluster0.oe4kfy.mongodb.net

localhost:27017

- admin
 - temproles
 - tempusers
- config
 - settings
- culibspace
 - user**
- local
 - replsetelection
 - replsetminvalid
 - startup_log
- test
 - post
 - postl
 - post_w_options

localhost:27017 > culibspace > user

Documents 4 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

[EXPLAIN](#) [RESET](#) [FIND](#) [OPTIONS](#)

[ADD DATA](#) [UPDATE](#) [DELETE](#) [EXPORT DATA](#) [EXPORT CODE](#)

25 1-4 of 4

```
>_MONGOSH
> function getBadUsersReport() {
  return db.user.aggregate([
    {
      $addFields: {
        total_failed_bookings: {
          $size: {
            $filter: {
              input: "$bookings",
              as: "b",
              cond: { $in: ["$b.Status", ["No-show", "Cancelled"]] }
            }
          }
        },
        total_success_bookings: {
          $size: {
            $filter: {
              input: "$bookings",
              as: "b",
              cond: { $eq: ["$b.Status", "Completed"] }
            }
          }
        }
      }
    },
    {
      $match: {
        $or: [
          { Behaviour_Score: { $lt: 100 } },
          { total_failed_bookings: { $gte: 1 } }
        ]
      }
    },
    {
      $project: {
        _id: 0,
        UID: 1,
        Firstname: 1,
        Lastname: 1,
        Phone: 1,
        Email: 1,
        Behaviour_Score: 1,
        User_type: 1,
        total_failed_bookings: 1,
        total_success_bookings: 1
      }
    }
  ])
}
```

The screenshot shows the MongoDB Compass application. On the left sidebar, the 'CONNECTIONS (2)' section is expanded, showing a local connection 'localhost:27017'. Under this connection, the 'user' database is selected. The main panel displays a MongoDB query in the 'user' collection. The query is a JavaScript function that iterates over a list of user objects and calls the 'getBadUsersReport()' function for each. The query is as follows:

```
>_MONGODB
[{"_id": 0,
  UID: 1,
  Fullname: { $concat: ["$Firstname", " ", "$Lastname"] },
  User_type: 1,
  Behaviour_Score: 1,
  total_failed_bookings: 1,
  total_success_bookings: 1
},
{
  $sort: {
    total_failed_bookings: -1,
    Behaviour_Score: 1
  }
}
]);
< [Function: getBadUsersReport]
> getBadUsersReport()
< {
  UID: 5,
  Behaviour_Score: 95,
  User_type: 'UNIVERSITY',
  total_failed_bookings: 1,
  total_success_bookings: 0,
  Fullname: 'Eve Wilson'
}
{
  UID: 3,
  Behaviour_Score: 100,
  User_type: 'THAI',
  total_failed_bookings: 1,
  total_success_bookings: 0,
  Fullname: 'Carol Lee'
}
]
culibspace>
```